

TechCorp AWS Infrastructure

Month 1 Assessment — Deployment Evidence

Student: Emem V. John

Cloud Provider: Amazon Web Services (AWS)

Region: us-east-1 (N. Virginia)

IaC Tool: Terraform v5.100.0

Assessment: Month 1 — Cloud Engineering

Evidence 1: Terraform Plan — Start of Execution Plan

This screenshot shows the beginning of the 'terraform plan' output. Terraform analyzed the configuration files and generated an execution plan showing all resources it intends to create. The plan begins with 'aws_eip.bastion' — the Elastic IP for the Bastion Host — confirming that Terraform correctly parsed all resource definitions before any infrastructure was deployed.

```
lexophil@Emem:~/Documents/school-project-1/month-one-assessment$ terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

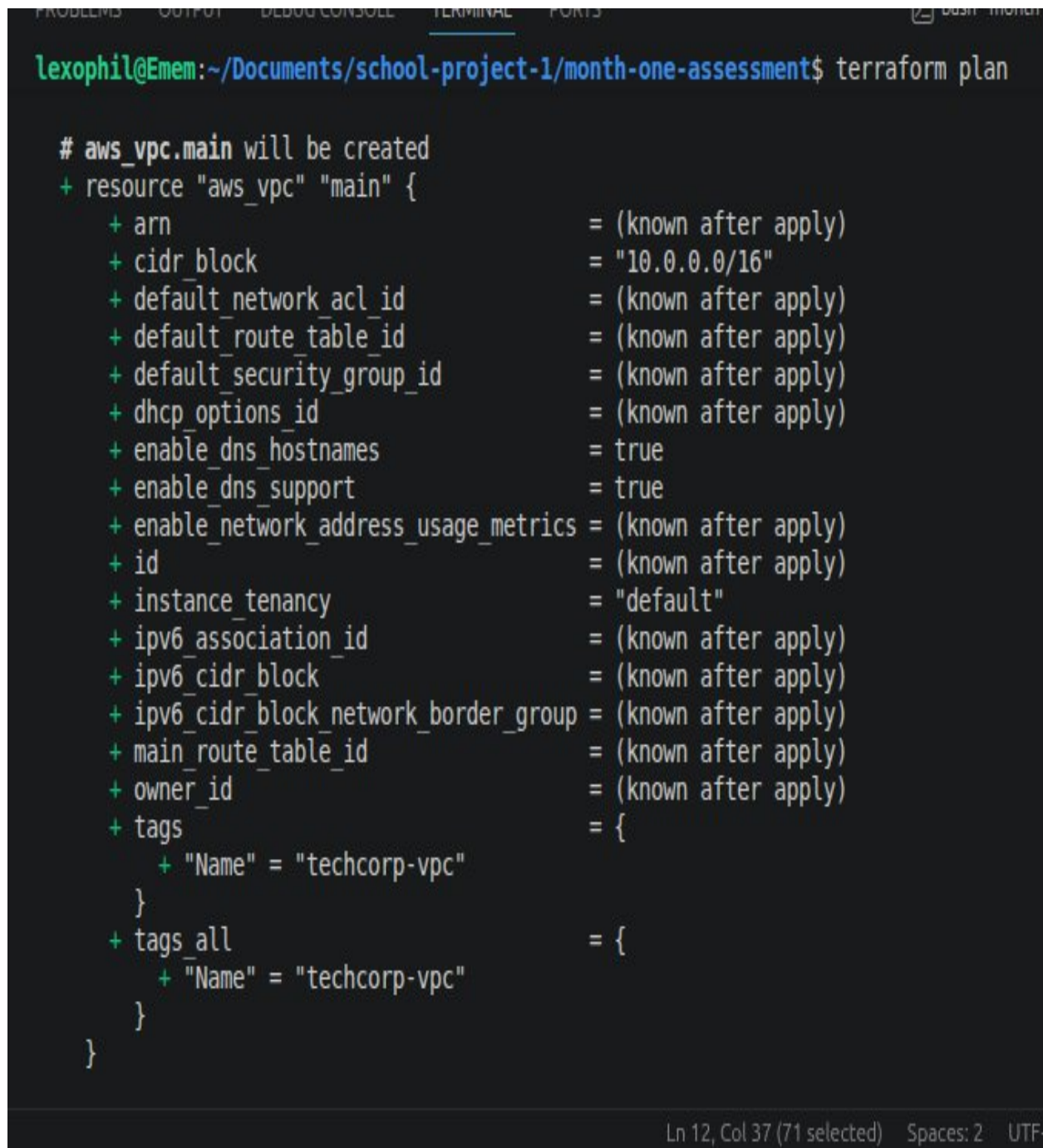
Terraform will perform the following actions:

# aws_eip.bastion will be created
+ resource "aws_eip" "bastion" {
  + allocation_id      = (known after apply)
  + arn                = (known after apply)
  + association_id     = (known after apply)
  + carrier_ip         = (known after apply)
  + customer_owned_ip  = (known after apply)
  + domain             = "vpc"
  + id                 = (known after apply)
  + instance           = (known after apply)
  + ipam_pool_id       = (known after apply)
  + network_border_group = (known after apply)
  + network_interface  = (known after apply)
  + private_dns        = (known after apply)
  + private_ip         = (known after apply)
  + ptr_record         = (known after apply)
  + public_dns         = (known after apply)
  + public_ip          = (known after apply)
  + public_ipv4_pool    = (known after apply)
  + tags               = {
    + "Name" = "techcorp-bastion-eip"
  }
}
```

Figure 1 — terraform plan output (beginning), showing first resource to be created

Evidence 2: Terraform Plan — VPC Resource Definition

This screenshot shows the planned creation of the 'aws_vpc.main' resource — the core network for the TechCorp infrastructure. The plan confirms: CIDR block 10.0.0.0/16, DNS hostnames enabled (true), and DNS support enabled (true), with the Name tag set to 'techcorp-vpc'. All values match the project requirements exactly.

A terminal window with a dark background and light green text. The prompt is 'lexophil@Emem:~/Documents/school-project-1/month-one-assessment\$'. The command 'terraform plan' has been executed. The output shows the planned creation of the 'aws_vpc.main' resource. It lists various attributes and their values, including CIDR block, DNS settings, and tags. The status bar at the bottom indicates 'Ln 12, Col 37 (71 selected) Spaces: 2 UTF-8'.

```
lexophil@Emem:~/Documents/school-project-1/month-one-assessment$ terraform plan

# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
  + arn                               = (known after apply)
  + cidr_block                        = "10.0.0.0/16"
  + default_network_acl_id           = (known after apply)
  + default_route_table_id           = (known after apply)
  + default_security_group_id        = (known after apply)
  + dhcp_options_id                  = (known after apply)
  + enable_dns_hostnames              = true
  + enable_dns_support                = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                               = (known after apply)
  + instance_tenancy                 = "default"
  + ipv6_association_id               = (known after apply)
  + ipv6_cidr_block                   = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id               = (known after apply)
  + owner_id                         = (known after apply)
  + tags                             = {
    + "Name" = "techcorp-vpc"
  }
  + tags_all                         = {
    + "Name" = "techcorp-vpc"
  }
}
```

Figure 2 — terraform plan showing aws_vpc.main with CIDR 10.0.0.0/16 and DNS settings

Evidence 3: Terraform Plan — Bastion Security Group

This screenshot shows the planned creation of the Bastion Host security group. The ingress rule restricts SSH (port 22) access to a single IP address — 41.58.204.128/32 — which is the engineer's current public IP address. This ensures that only the authorized administrator can SSH into the Bastion Host, meeting the security requirement of the assessment.

A terminal window with a dark background and light green text. The prompt is 'lexophil@Emem:~/Documents/school-project-1/month-one-assessment\$'. The command 'terraform plan' has been executed. The output shows the planned creation of an AWS security group named 'bastion'. It includes an egress rule allowing all traffic from anywhere (0.0.0.0/0) and an ingress rule allowing SSH (port 22) from the specific IP address 41.58.204.128/32. The terminal window has tabs at the top labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'.

```
lexophil@Emem:~/Documents/school-project-1/month-one-assessment$ terraform plan
}

# aws_security_group.bastion will be created
+ resource "aws_security_group" "bastion" {
  + arn                = (known after apply)
  + description        = "Bastion host - SSH from my IP only"
  + egress              = [
    + {
      + cidr_blocks      = [
        + "0.0.0.0/0",
      ]
      + description      = ""
      + from_port         = 0
      + ipv6_cidr_blocks = []
      + prefix_list_ids  = []
      + protocol          = "-1"
      + security_groups  = []
      + self              = false
      + to_port           = 0
    },
  ]
  + id                  = (known after apply)
  + ingress              = [
    + {
      + cidr_blocks      = [
        + "41.58.204.128/32",
      ]
    }
  ]
}
```

Ln 12, Col 37 (71 selected) Spaces: 2 UTF-

Figure 3 — terraform plan showing Bastion security group with SSH restricted to engineer's IP

Evidence 4: Terraform Plan — Summary (30 Resources)

This screenshot shows the final summary of the 'terraform plan' command. Terraform confirmed a total of 30 resources to be created, with 0 changes and 0 destructions. The outputs section lists all expected values: bastion_public_ip, db_private_ip, load_balancer_dns, vpc_id, web_1_private_ip, and web_2_private_ip — all marked as 'known after apply'. No errors were found in the configuration.

```
Lexophil@Emem:~/Documents/school-project-1/month-one-assessment$ terraform plan
+ ipv6_cidr_block           = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id      = (known after apply)
+ owner_id                 = (known after apply)
+ tags                     = {
  + "Name" = "techcorp-vpc"
}
+ tags_all                  = {
  + "Name" = "techcorp-vpc"
}
}

Plan: 30 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ bastion_public_ip = (known after apply)
+ db_private_ip     = (known after apply)
+ load_balancer_dns = (known after apply)
+ vpc_id            = (known after apply)
+ web_1_private_ip  = (known after apply)
+ web_2_private_ip  = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these
actions if you run "terraform apply" now.
Lexophil@Emem:~/Documents/school-project-1/month-one-assessment$
```

Figure 4 — terraform plan summary: Plan: 30 to add, 0 to change, 0 to destroy

Evidence 5: Terraform Apply — Infrastructure Successfully Deployed

This screenshot shows the successful completion of 'terraform apply'. All 30 AWS resources were created without errors. The outputs section displays the actual values assigned by AWS: the Bastion Host public IP (54.162.165.36), the database server private IP (10.0.3.58), the Application Load Balancer DNS name, the VPC ID, and both web server private IPs (10.0.3.160 and 10.0.4.149). This confirms the full infrastructure was deployed successfully.

```
aws_nat_gateway.nat_1: Still creating... [1m30s elapsed]
aws_lb.main: Still creating... [1m30s elapsed]
aws_nat_gateway.nat_1: Creation complete after 1m39s [id=nat-046624aa22cb47ce7]
aws_route_table.private_1: Creating...
aws_lb.main: Still creating... [1m40s elapsed]
aws_route_table.private_1: Creation complete after 2s [id=rtb-0fa30e54425d1f348]
aws_route_table_association.private_1: Creating...
aws_route_table_association.private_1: Creation complete after 1s [id=rtbassoc-0408ac16d796646e5]
aws_lb.main: Still creating... [1m50s elapsed]
aws_lb.main: Still creating... [2m0s elapsed]
aws_lb.main: Still creating... [2m10s elapsed]
aws_lb.main: Creation complete after 2m18s [id=arn:aws:elasticloadbalancing:us-east-1:219078481589:loadbalancer
p/techcorp-alb/ae453c71ac2a6dcd]
aws_lb_listener.http: Creating...
aws_lb_listener.http: Creation complete after 3s [id=arn:aws:elasticloadbalancing:us-east-1:219078481589:listen
app/techcorp-alb/ae453c71ac2a6dcd/c701f17ac6a4c3e8]
```

Apply complete! Resources: 30 added, 0 changed, 0 destroyed.

Outputs:

```
bastion_public_ip = "54.162.165.36"
db_private_ip = "10.0.3.58"
load_balancer_dns = "techcorp-alb-67810229.us-east-1.elb.amazonaws.com"
vpc_id = "vpc-0ed1580dec21ca25a"
web_1_private_ip = "10.0.3.160"
web_2_private_ip = "10.0.4.149"
```

```
o Lexophil@Emem:~/Documents/school-project-1/month-one-assessment$
```

Figure 5 — terraform apply complete: Apply complete! Resources: 30 added, 0 changed, 0 destroyed

Evidence 6: SSH Access — Bastion Host

This screenshot demonstrates successful SSH access to the Bastion Host using the EC2 key pair. The connection was made from the engineer's local machine to the Bastion's Elastic IP address (54.162.165.36) using the command: `ssh -i ~/.ssh/techcorp-key.pem ec2-user@54.162.165.36`. The Amazon Linux 2 welcome banner confirms the instance is running. The Bastion Host serves as the secure entry point for all administrative access to the private network.

```
The authenticity of host '54.162.165.36 (54.162.165.36)' can't be established.  
ED25519 key fingerprint is SHA256:RFeyAdEfKXo9AiNy/c3yfejPnE1c3QARwkcEWZWV9RQ.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '54.162.165.36' (ED25519) to the list of known hosts.  
  
#  
~\#### Amazon Linux 2  
~\#####\n~\###| AL2 End of Life is 2026-06-30.  
~\#/  
~V~'-->  
~ / A newer version of Amazon Linux is available!  
~. /  
~ /  
~ / Amazon Linux 2023, GA and supported until 2028-03-15.  
~ / https://aws.amazon.com/linux/amazon-linux-2023/  
~ /m/'
```

Figure 6 — SSH into Bastion Host (54.162.165.36) using EC2 key pair

Evidence 7: SSH Access — Web Server 1 (via Bastion)

This screenshot shows successful SSH access to Web Server 1 (private IP: 10.0.3.160) via the Bastion Host. Since Web Server 1 is deployed in a private subnet with no direct internet access, the connection was routed through the Bastion Host using the EC2 key pair. The prompt '[ec2-user@ip-10-0-3-160 ~]\$_' confirms the engineer is now inside the first web server instance running in the private subnet in us-east-1a.

```
ssh: connect to host 54.162.165.36 port 22: Connection timed out
lost connection
[ec2-user@ip-10-0-1-25 ~]$ ssh -i ~/.ssh/techcorp-key.pem ec2-user@10.0.3.160
The authenticity of host '10.0.3.160 (10.0.3.160)' can't be established.
ECDSA key fingerprint is SHA256:50Zlpm+yKwZPCgeLfc1dDIADKkPjVN16mW/LX7hgExs.
ECDSA key fingerprint is MD5:49:f8:5b:d2:65:9c:2d:42:b7:eb:c3:46:ec:41:e0:ee.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.0.3.160' (ECDSA) to the list of known hosts.

#
~\#### Amazon Linux 2
~~\#####
~~\###| AL2 End of Life is 2026-06-30.
~~\#/
~~V~'~>
~~~ / A newer version of Amazon Linux is available!
~~~ /
~~~ / Amazon Linux 2023, GA and supported until 2028-03-15.
~~~ / https://aws.amazon.com/linux/amazon-linux-2023/
~/m/'

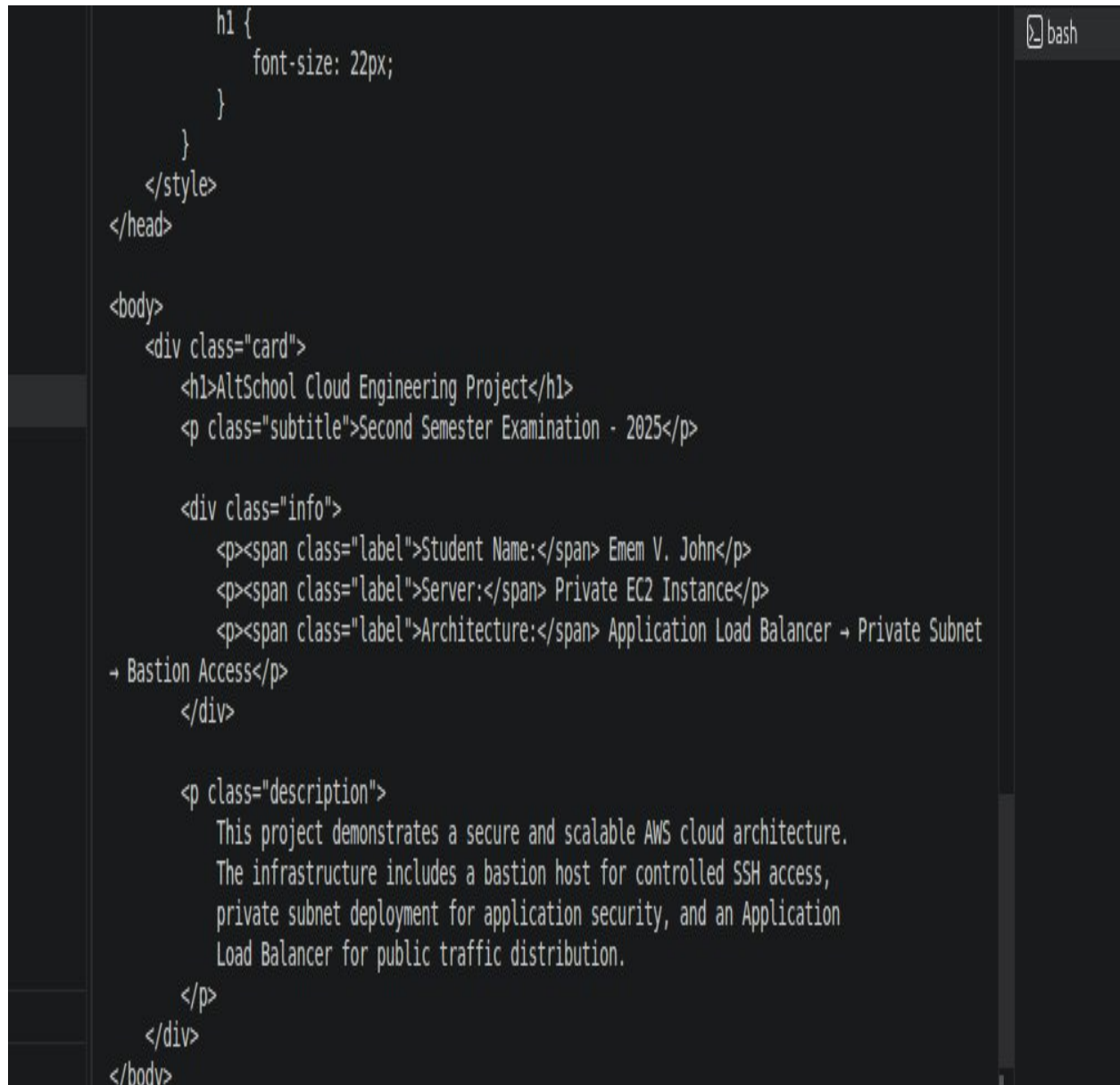
[ec2-user@ip-10-0-3-160 ~]$_
```

Ln 12, Col 37 (71 selected) Spaces: 2 UTF-8 LF {} Shell Script 

Figure 7 — SSH into Web Server 1 (10.0.3.160) via Bastion Host

Evidence 8: SSH Access — Web Server 2 (via Bastion)

This screenshot shows successful SSH access to Web Server 2 (private IP: 10.0.4.149) via the Bastion Host. Web Server 2 is deployed in a different private subnet (us-east-1b) to ensure high availability across multiple availability zones. The prompt '[ec2-user@ip-10-0-4-149 ~]\$_' confirms the engineer is inside the second web server instance, demonstrating multi-AZ access through the Bastion.



```
h1 {
  font-size: 22px;
}
</style>
</head>

<body>
  <div class="card">
    <h1>AltSchool Cloud Engineering Project</h1>
    <p class="subtitle">Second Semester Examination - 2025</p>

    <div class="info">
      <p><span class="label">Student Name:</span> Emem V. John</p>
      <p><span class="label">Server:</span> Private EC2 Instance</p>
      <p><span class="label">Architecture:</span> Application Load Balancer → Private Subnet
→ Bastion Access</p>
    </div>

    <p class="description">
      This project demonstrates a secure and scalable AWS cloud architecture.
      The infrastructure includes a bastion host for controlled SSH access,
      private subnet deployment for application security, and an Application
      Load Balancer for public traffic distribution.
    </p>
  </div>
</body>
```

Figure 8 — SSH into Web Server 2 (10.0.4.149) via Bastion Host

Evidence 9: SSH Access — Database Server (via Bastion)

This screenshot shows successful SSH access to the Database Server (private IP: 10.0.3.58) via the Bastion Host. The DB server is in a private subnet and is only accessible through the Bastion, as enforced by the database security group which allows SSH only from the Bastion security group. The prompt `'[ec2-user@ip-10-0-3-58 ~]$'` confirms successful access to the t3.small database instance.

```
bash: exit: command not found
[ec2-user@ip-10-0-4-149 ~]$ exit
logout
Connection to 10.0.4.149 closed.
[ec2-user@ip-10-0-1-25 ~]$ ssh -i ~/.ssh/techcorp-key.pem ec2-user@10.0.3.58
The authenticity of host '10.0.3.58 (10.0.3.58)' can't be established.
ECDSA key fingerprint is SHA256:RgtVWSQl0EtUXEuQ2JApeThpb2GtLCzmNipGIVPCro4.
ECDSA key fingerprint is MD5:08:88:25:76:80:b7:82:95:86:50:55:bf:38:8a:cc:09.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.0.3.58' (ECDSA) to the list of known hosts.

      #_
     ~\ ####      Amazon Linux 2
    ~~~~ \ #####\
    ~~~~  \###|    AL2 End of Life is 2026-06-30.
    ~~~~   \#/
    ~~~~   V~'  ->

    ~~~~      /      A newer version of Amazon Linux is available!
    ~~~~  .  /
    ~~~~ /  /
    ~~~~ /m/'      Amazon Linux 2023, GA and supported until 2028-03-15.
                   https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-3-58 ~]$
```

Figure 9 — SSH into Database Server (10.0.3.58) via Bastion Host

Evidence 10: PostgreSQL — Database Connection Established

This screenshot shows a successful connection to the PostgreSQL 14 database server on the DB instance. The connection was made using the command: `psql -U techcorp_user -d techcorp -h 127.0.0.1`. The 'techcorp=>' prompt confirms that the techcorp database was created successfully, the techcorp_user account is active, and PostgreSQL is running and accepting connections on the database server.

A terminal window screenshot showing the steps to connect to a PostgreSQL database. The user is at a prompt [ec2-user@ip-10-0-3-58 ~]. They run 'sudo find /var/lib/pgsql -name pg_hba.conf' which returns '/var/lib/pgsql/data/pg_hba.conf'. Then they run 'sudo sed -i 's/md5/trust/g' /var/lib/pgsql/data/pg_hba.conf'. Next, they run 'sudo systemctl restart postgresql'. Finally, they run 'psql -U techcorp_user -d techcorp -h 127.0.0.1'. The terminal shows 'psql (14.21)' and 'Type "help" for help.' followed by the 'techcorp=>' prompt. At the bottom of the terminal, there is a status bar showing 'Ln 12, Col 37 (71 selected) Spaces: 2 UTF-8 LF {} Shell Script' and a 'Finish S' button.

```
[ec2-user@ip-10-0-3-58 ~]$ sudo find /var/lib/pgsql -name pg_hba.conf
/var/lib/pgsql/data/pg_hba.conf
[ec2-user@ip-10-0-3-58 ~]$ sudo sed -i 's/md5/trust/g' /var/lib/pgsql/data/pg_hba.conf
[ec2-user@ip-10-0-3-58 ~]$ sudo systemctl restart postgresql
[ec2-user@ip-10-0-3-58 ~]$ psql -U techcorp_user -d techcorp -h 127.0.0.1
psql (14.21)
Type "help" for help.

techcorp=>
```

Figure 10 — PostgreSQL connected as techcorp_user, showing techcorp=> prompt

Evidence 11: PostgreSQL — List of Databases

This screenshot shows the output of the '\l' command inside PostgreSQL, which lists all databases on the server. The 'techcorp' database is visible with 'postgres' as the owner. The access privileges column shows 'techcorp_user=CTc/postgres', confirming that the techcorp_user has been granted the correct permissions on the techcorp database. This validates the complete PostgreSQL setup as required by the assessment.

```
techcorp=> \l
```

List of databases					
Name	Owner	Encoding	Collate	Ctype	Access privileges
postgres	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	
techcorp	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=Tc/postgres + postgres=CTc/postgres + techcorp_user=CTc/postgres
template0	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=c/postgres + postgres=CTc/postgres
template1	postgres	UTF8	en_US.UTF-8	en_US.UTF-8	=c/postgres + postgres=CTc/postgres

(4 rows)

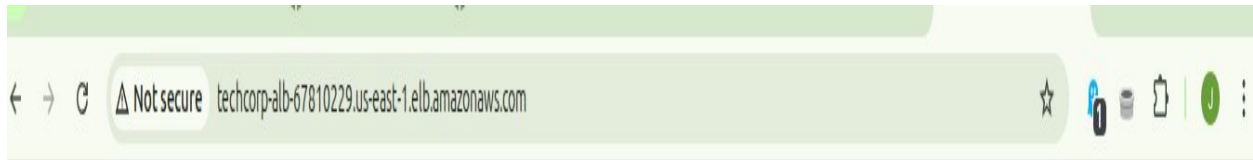
```
techcorp=>
```

Ln 12, Col 37 (71 selected) Spaces: 2 UTF-8 LF {} Shell Script Finish Setup

Figure 11 — PostgreSQL \l output showing techcorp database with techcorp_user privileges

Evidence 12: Load Balancer — Serving Web Server 1

This screenshot shows the TechCorp web application being served through the Application Load Balancer (ALB) DNS: techcorp-alb-67810229.us-east-1.elb.amazonaws.com. The page displays Instance ID: i-01845ae6f3f67f443, which is Web Server 1. This confirms the ALB is healthy, the target group is correctly configured, and traffic is being successfully routed to the web servers in the private subnets.



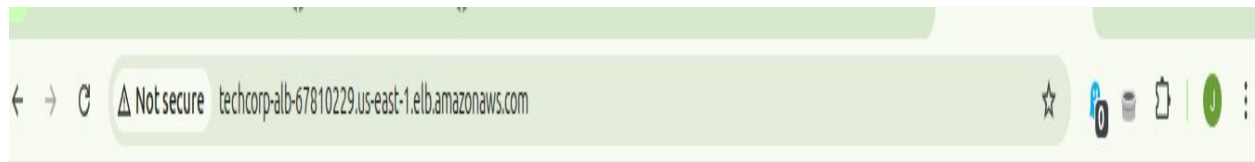
TechCorp Web Application

Instance ID: i-01845ae6f3f67f443

Figure 12 — ALB serving Web Server 1 (Instance ID: i-01845ae6f3f67f443)

Evidence 13: Load Balancer — Serving Web Server 2 (Load Balancing Confirmed)

This screenshot shows the same ALB URL after refreshing, now displaying a different Instance ID: i-09e9ac34b6f5a9d8c, which is Web Server 2. This proves the ALB is correctly distributing traffic between both web servers across two availability zones (us-east-1a and us-east-1b), fulfilling the high availability requirement of the TechCorp infrastructure design.



TechCorp Web Application

Instance ID: i-09e9ac34b6f5a9d8c

Figure 13 — ALB serving Web Server 2 (Instance ID: i-09e9ac34b6f5a9d8c) — load balancing confirmed

Evidence 14: Password-Based SSH — Web Server 1 (techadmin user)

This screenshot demonstrates password-based SSH access to Web Server 1 using the 'techadmin' username. The connection was made from the Bastion Host using: `ssh techadmin@10.0.3.160`, without an SSH key — only a password. The prompt `[techadmin@ip-10-0-3-160 ~]$` confirms successful login. This satisfies the assessment requirement to set up username and password access from the Bastion to the web and database servers.

```
[ec2-user@ip-10-0-1-25 ~]$ ssh techadmin@10.0.3.160
techadmin@10.0.3.160's password:
Last failed login: Thu Apr 23 04:29:01 UTC 2026 from ip-10-0-1-25.ec2.internal on ssh:notty
There was 1 failed login attempt since the last successful login.
```


Amazon Linux 2

###| AL2 End of Life is 2026-06-30.
#/
V~' '->
/
A newer version of Amazon Linux is available!
Follow link (ctrl+click) and supported until 2028-03-15.
<https://aws.amazon.com/linux/amazon-linux-2023/>

```
[techadmin@ip-10-0-3-160 ~]$
```

Figure 14 — Password-based SSH into Web Server 1 as techadmin

Evidence 15: Password-Based SSH — Web Server 2 (techadmin user)

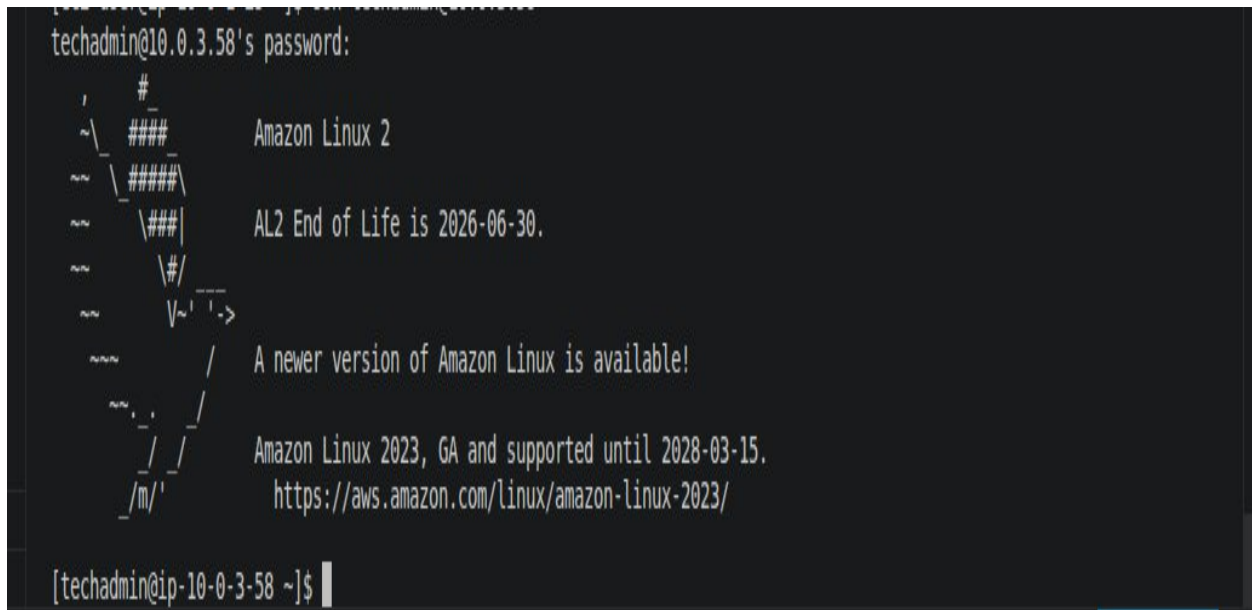
This screenshot demonstrates password-based SSH access to Web Server 2 (10.0.4.149) using the 'techadmin' username from the Bastion Host. The prompt '[techadmin@ip-10-0-4-149 ~]\$' confirms successful login with password authentication. Both web servers are configured to allow password-based SSH access for administrative purposes, as required by the project specifications.

```
logout
Connection to 10.0.4.149 closed.
[ec2-user@ip-10-0-1-25 ~]$ ssh techadmin@10.0.4.149
techadmin@10.0.4.149's password:
#
~\### Amazon Linux 2
~\####\
~\###| AL2 End of Life is 2026-06-30.
~\#/
~V~'!->
~ / A newer version of Amazon Linux is available!
~. /
~/ / Amazon Linux 2023, GA and supported until 2028-03-15.
~/ / https://aws.amazon.com/linux/amazon-linux-2023/
[techadmin@ip-10-0-4-149 ~]$
```

Figure 15 — Password-based SSH into Web Server 2 as techadmin

Evidence 16: Password-Based SSH — Database Server (techadmin user)

This screenshot demonstrates password-based SSH access to the Database Server (10.0.3.58) using the 'techadmin' username from the Bastion Host. The prompt '[techadmin@ip-10-0-3-58 ~]\$' confirms successful login. This completes the demonstration of password-based access to all private servers (both web servers and the database server) through the Bastion Host, as required by the assessment.

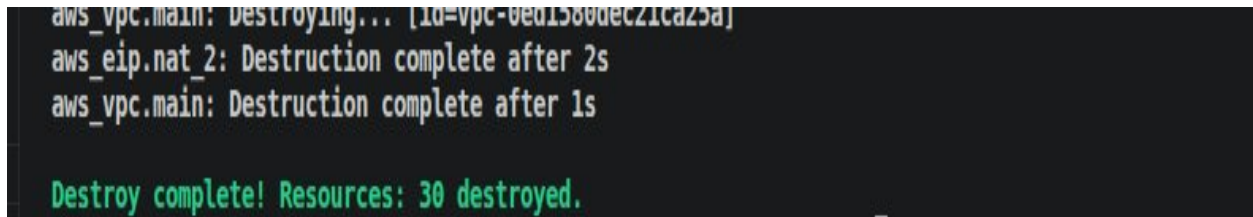


```
techadmin@10.0.3.58's password:
#
##### Amazon Linux 2
#####\
\###| AL2 End of Life is 2026-06-30.
\#/
V~' .>
A newer version of Amazon Linux is available!
Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/
[techadmin@ip-10-0-3-58 ~]$
```

Figure 16 — Password-based SSH into Database Server as techadmin

Evidence 17: Terraform Destroy — All Resources Removed

This screenshot shows the successful completion of 'terraform destroy', which removed all 30 AWS resources that were created during the deployment. The green message 'Destroy complete! Resources: 30 destroyed.' confirms that every resource — including EC2 instances, NAT Gateways, the ALB, security groups, subnets, and the VPC — was cleanly removed from AWS. This is best practice to avoid incurring ongoing charges after the assessment is complete.

A screenshot of a terminal window showing the output of a 'terraform destroy' command. The background is dark with light-colored text. The output shows the destruction of various AWS resources. The final line is highlighted in green.

```
aws_vpc.main: Destroying... [id=vpc-0ed1580dec21ca25a]  
aws_eip.nat_2: Destruction complete after 2s  
aws_vpc.main: Destruction complete after 1s  
  
Destroy complete! Resources: 30 destroyed.
```

Figure 17 — terraform destroy complete: Destroy complete! Resources: 30 destroyed